

Reinforcement Learning for Optimal Drift Rate Control in Brownian Processing Systems

Yassin Ikrichi

Laboratory IR2M

Faculty of Sciences and Techniques Settat

Hassan 1st University

Settat, Morocco

yassinikrichipro@gmail.com

Abdelghani Bentahar

Laboratory IR2M

Faculty of Sciences and Techniques Settat

Hassan 1st University

Settat, Morocco

abdelghani.bentahar@uhp.ac.ma

Abstract—This paper studies the control of the drift rate in a Brownian queueing system with finite capacity. The system state evolves as a reflected diffusion process in the interval from 0 to b . The objective is to minimize the long-run average cost, which combines an operational effort cost and a rejection penalty when the system reaches its capacity. This problem was analytically solved by Ata, Harrison, and Shepp (2005) [1] using stochastic control theory.

In this work, we explore a model-free approach based on Deep Reinforcement Learning. In particular, we use the Proximal Policy Optimization (PPO) algorithm to learn the control policy directly from interaction with the system, without using the analytical solution or the Bellman equation. The stochastic dynamics are simulated with an Euler–Maruyama scheme that reproduces the Brownian motion and the reflection at the boundaries.

The numerical results show that the learned policy has a smooth and continuous structure. In particular, the agent discovers a costless region where no effort is applied when the queue level is low. Near the system capacity, the learned actions remain bounded and increase gradually, reflecting a practical trade-off between energy cost and rejection penalties. These results show that reinforcement learning can recover the main structure of the optimal control policy and provide a useful approach for more complex stochastic control problems where analytical solutions are not available.

Index Terms—Reinforcement learning, stochastic control, reflected Brownian motion, finite queue, PPO.

I. INTRODUCTION

Finite-capacity queueing processing systems (e.g., network servers or cloud platforms) must dynamically manage a trade-off between processing effort and task rejection. In these systems, the queue content $Z(t)$ evolves within a bounded interval $[0, b]$ under the effect of Brownian noise and a negative drift control (service) applied by the manager. The classical mathematical modeling (cf. Ata *et al.* [1]) describes Z by a reflected diffusion:

$$dZ(t) = dX(t) - \theta(Z(t))dt + dL(t) - dU(t), \quad (1)$$

where $X(t)$ is a Brownian motion with variance σ^2 , and $L(t), U(t)$ are increasing processes that cause $Z(t)$ to bounce at the lower bound 0 and upper bound b , respectively. The manager chooses the state-dependent effort drift $\theta(z) \geq 0$.

This work was supported by the Moroccan Centre National pour la Recherche Scientifique et Technique (CNRST).

The cumulative cost includes two terms: a continuous cost $c(\theta)$ (increasing with the effort) and a penalty p per unit of rejection (increase of U) at the upper boundary. Objective: minimize the long-term average cost vs. capacity constraints. Notably, Ata *et al.* explicitly solved this problem by obtaining an optimal multi-threshold policy [1]: the closer $Z(t)$ approaches b , the more θ is increased; otherwise, the effort is relaxed toward 0. This threshold HJB solution is a fundamental theoretical result for controlled Brownian systems [1].

However, analytically solving such problems is often impossible in practice for more complex models. This is why we explore here a *data-driven* alternative: Reinforcement Learning (RL). Reinforcement learning allows learning an optimal policy without knowing the system equations in advance. Previous works (Sutton & Barto [2], and others) have shown that RL is effective for complex stochastic control tasks. We therefore seek to see if a DRL agent can autonomously recover the threshold policy by interacting with a simulator of the reflected SDE. Concretely, we use an Euler–Maruyama scheme to generate the trajectory Z_{n+1} from Z_n and a continuous or discrete action θ_n , then the agent receives the reward $r_n = -(c(\theta_n)\Delta t + p \cdot \max(0, Z_{n+1} - b))$.

The following paragraph reviews the state of the art. Ata *et al.* [1] established the theoretical foundation with the optimal solution. In the RL domain, recent algorithms like PPO, SAC, or TD3 have proven powerful for the continuous control of unknown systems. We will also mention how these methods can be adapted to the Brownian problem.

Our main goal is to show that Reinforcement Learning can solve this specific control problem. By using the PPO algorithm, we can find an optimal policy that is very close to the mathematical theory. We aim to prove that an agent can learn the best behavior through interaction, even without knowing the underlying equations.

The paper is organized as follows. Section II describes the system model. In Section III, we explain how we use PPO and how we prepare the data for the agent. Section IV shows the results and compares the AI policy with the theoretical solution.

II. PRELIMINARIES AND SYSTEM MODEL

Our work uses the control model from the paper by Ata, Harrison, and Shepp (2005) [1]. We use their math formulas to create the environment. This is where our Reinforcement Learning agent learns and operates.

A. System Dynamics

The system state at time t , denoted by $Z(t)$, represents the buffer content (or queue length) and is constrained within a finite interval $[0, b]$, where b represents the maximum system capacity. The evolution of this state trajectory is governed by the following stochastic differential equation:

$$Z(t) = Z(0) + X(t) - \int_0^t \theta(Z(s))ds + L(t) - U(t) \quad (2)$$

The variable $Z(0)$ defines the initial system state. We denote by $X = \{X(t), t \geq 0\}$ a Brownian motion with variance σ^2 . This term models stochastic fluctuations, such as variability in arrival and service processes. The control policy $\theta(Z(s))$ acts as a state-dependent drift. Physically, this represents the service effort used to shorten the queue. The state must remain within $[0, b]$. We introduce two regulation processes L and U , to enforce these boundaries. The lower regulator L is a non-decreasing process that prevents $Z(t)$ from becoming negative. It satisfies the condition $\int_0^\infty Z(t)dL(t) = 0$. This integral ensures that L increases only when the state hits the boundary $Z(t) = 0$. We assume this reflection is cost-free and the upper regulator U prevents the state from exceeding the buffer capacity b . It follows a similar logic $\int_0^\infty (b - Z(t))dU(t) = 0$. This condition means U increases only at the boundary $Z(t) = b$. In our model, any increase in U represents rejected workload due to buffer overflow.

B. Cost Structure and Geometric Intuition

The objective of the control problem is to determine a policy $\theta(\cdot)$ that minimizes the long-run average cost. This objective function is defined by a trade-off between two competing costs: a continuous operational cost rate $c(\theta)$ associated with maintaining a specific service speed, and a fixed penalty cost p incurred for every unit of displacement of the upper boundary U (i.e., every rejection).

While Ata et al. [1] derive an explicit analytical solution for this problem, we focus on the geometric intuition of the optimal strategy to guide our learning approach. The control θ can be viewed as a downward restorative force modulated by the agent. A policy that is too weak will allow the trajectory to frequently impact the upper boundary b , accumulating substantial penalty costs p . On the other hand, a policy that exerts maximum force continuously will suppress the state near the lower boundary 0. Although this eliminates rejection penalties, it results in excessive and unnecessary operational expenditure.

Therefore, the optimal control strategy requires dynamic modulation [1]: the service effort should be relaxed near the lower boundary to allow for cost-free fluctuations, and

intensified progressively as the state approaches the capacity limit b to prevent costly rejections.

III. PROPOSED METHODOLOGY: DEEP REINFORCEMENT LEARNING

Although the stochastic control model presented in the previous section has an exact analytical solution, solving it relies on a complex second-order Bellman differential equation [11]. Our objective is to demonstrate that a Model-Free Deep Reinforcement Learning (RL) agent can empirically recover this strict optimal policy, paving the way for solving multi-dimensional systems where classical mathematics fail.

To achieve this, we reformulate the continuous Brownian control problem into a discrete-time Markov Decision Process (MDP), solved using the Proximal Policy Optimization (PPO) algorithm.

A. State and Action Space Formulation

To stabilize the training of neural networks, the physical system's spaces are mapped to normalized learning spaces [12]:

- **State space (Observation):** The system state $Z(t) \in [0, b]$ represents the queue level. At each time step n , the agent observes a normalized version of this state $s_n = Z_n/b \in [0, 1]$.
- **Action space:** The agent selects a continuous action corresponding to the negative drift rate (the effort) $\theta_n \in [0, \theta_{\max}]$, which is applied over a discrete time step Δt .

B. Discretized System Dynamics and Boundaries

The stochastic evolution of the system is simulated using a projected Euler-Maruyama scheme, which accurately reproduces the Brownian dynamics and the boundary reflection processes [6], [12]. At each step n , the update is performed in three phases:

1) *Unconstrained Proposal:* A candidate position $\tilde{Z}_{n+1}$ is calculated by combining the current state, the drift chosen by the agent, and the Brownian increment [4], [12]:

$$\tilde{Z}_{n+1} = Z_n - \theta_n \Delta t + \sigma \Delta W_n \quad (3)$$

where $\Delta W_n \sim \mathcal{N}(0, \Delta t)$ represents the random fluctuations with volatility σ .

2) *Rejection Quantification:* If the candidate trajectory exceeds the maximum capacity b , the surplus corresponds to the rejected work (the $U(t)$ process in the theory) [1], [14]:

$$\text{Reject}_n = \max(0, \tilde{Z}_{n+1} - b) \quad (4)$$

3) *Skorokhod Projection:* The final state is forced to remain within the admissible domain $[0, b]$ via a saturation function [1], [14]:

$$Z_{n+1} = \max(0, \min(b, \tilde{Z}_{n+1})) \quad (5)$$

This operation simultaneously captures the ceiling penalty and the costless displacement at the lower boundary $Z = 0$ ($L(t)$), since preventing the state from becoming negative does not generate any rejection value.

C. Reward Structure and Convex Cost

The reward function is the core element ensuring that the agent converges towards the optimal geometric policy of the theoretical model [1]. It consists of three components:

1) *Effort Cost (Strictly Convex)*: In accordance with the reference article's assumptions for obtaining smooth and continuous control, we model the operational cost $c(\cdot)$ as a strictly convex (quadratic) function of the drift rate [4]:

$$C_{\text{effort}} = c \cdot \theta_n^2 \cdot \Delta t \quad (6)$$

2) *Overflow Penalty*: Each unit of rejected work incurs a severe penalty proportional to the parameter p [4]:

$$C_{\text{reject}} = p \cdot \text{Reject}_n \quad (7)$$

3) *Reward Shaping*: To accelerate the algorithm's convergence given the scarcity of overflow events, we introduce a potential term $\Phi(Z) = -(Z/b)^2$ [3]. According to Ng et al.'s theorem (1999), adding a potential difference maintains the strict optimality of the policy:

$$F(Z_n, Z_{n+1}) = k \cdot (\Phi(Z_{n+1}) - \Phi(Z_n)) \quad (8)$$

The overall reward received by the agent at each transition is thus [4]:

$$r_n = -(C_{\text{effort}} + C_{\text{reject}}) + F(Z_n, Z_{n+1}) \quad (9)$$

D. PPO Optimization and Average Cost Criterion

The theoretical problem aims to minimize the long-term average cost (ergodic control), rather than a classic discounted cost commonly used in RL [5], [10].

To ensure our PPO (Proximal Policy Optimization) algorithm behaves ergodically while preserving the stability of advantage estimators, we train the model with a discount factor asymptotically close to 1 ($\gamma = 0.9999$) [5]. This setting forces the agent to consider a near-infinite time horizon. This is essential for it to learn to modulate its effort well before the queue reaches its capacity limit, thereby empirically discovering the costless threshold and the acceleration curve dictated by the Bellman equation.

The complete architecture of our proposed approach, summarizing the interaction between the PPO algorithm and the simulated physical environment, is illustrated in Fig. 1.

IV. RESULTS AND DISCUSSIONS

To evaluate the relevance of the Deep Reinforcement Learning (PPO) approach for solving the stochastic control problem, we analyzed the agent's behavior from three perspectives: learning convergence, the structure of the learned optimal policy, and the temporal dynamics in the presence of Brownian motion.

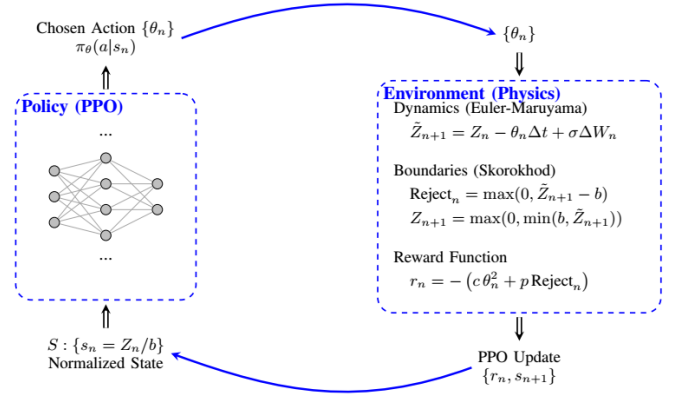


Fig. 1. Deep Reinforcement Learning architecture applied to stochastic control. The neural network (left) generates the effort policy θ_n , which is evaluated by the physical dynamics equations and boundary processes (right) to calculate the reward r_n .

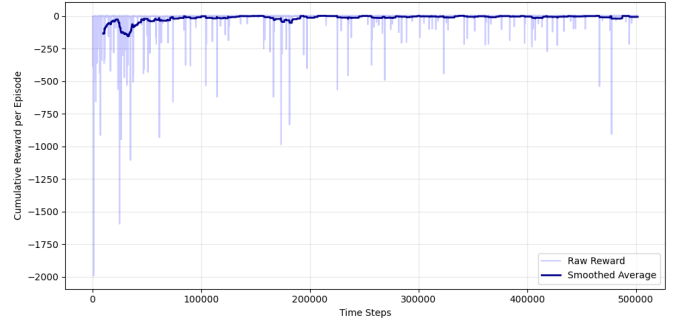


Fig. 2. Evolution of the average cumulative reward over 500,000 time steps.

A. Convergence and Algorithmic Stability

Fig. 2 shows how the reward changes during training. The agent finds an optimal policy after a short training period. You can still see some fluctuations on the graph, but this is normal. These movements are caused by the Brownian noise in our model. Even with this noise, the policy remains optimal. The agent has learned to handle the random shocks of the system while maintaining the best possible control.

B. Learned Policy Profile and Costless Region

One of the most significant results of this study is presented in Fig. 3, which plots the commanded effort θ as a function of the system state Z . To evaluate the agent's accuracy, we superimposed the exact theoretical optimal policy derived from the underlying differential equations. Without any prior knowledge of these equations, the PPO agent successfully reconstructed the optimal mathematical structure expected for a control problem with a strictly convex cost.

A costless region appears for $Z < 4.5$. In this interval, the agent sets the effort to zero $\theta = 0$. We note a structural difference between the theoretical and learned policies here. The analytical solution requires a positive effort for any $Z > 0$ because the derivative of the quadratic cost is zero at the origin.

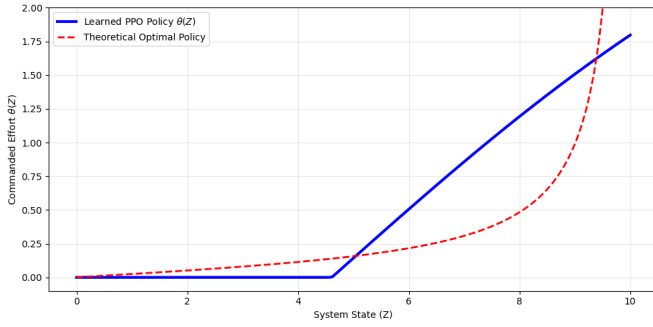


Fig. 3. Profile of the learned policy $\theta(Z)$ compared to the theoretical optimal policy, illustrating the emergence of a costless region.

The RL agent applies a sparse policy instead. The micro-rewards from infinitesimal efforts are lost in the stochastic noise. Therefore, the agent maintains absolute inaction when the overflow probability is minimal.

A second difference occurs near the capacity limit $Z \rightarrow 10$. The theoretical effort approaches infinity to strictly prevent boundary rejection. The RL agent bounds its response, peaking at approximately $\theta = 1.8$. The algorithm accepts a linear rejection penalty p instead of applying maximum effort. Pushing the effort to infinity forces the quadratic cost to explode, which contradicts the objective of minimizing the long-term average cost.

C. Control Dynamics and Skorokhod Boundaries

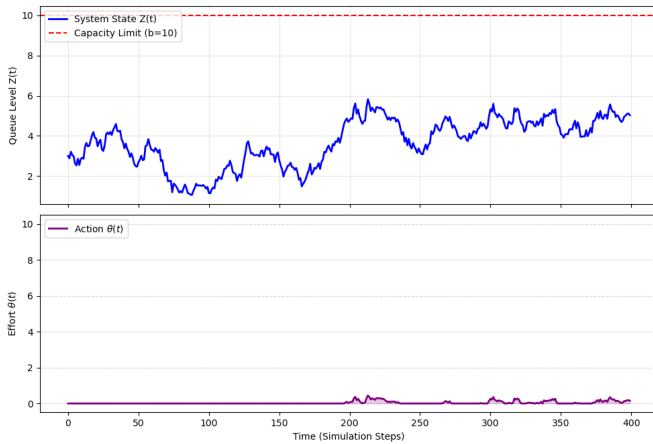


Fig. 4. Dynamics of the system state $Z(t)$ and the action $\theta(t)$ near the boundaries.

To validate the controller's real-time behavior, we simulated the learned policy against a stochastic trajectory (Fig. 4). The agent demonstrates excellent management of economic trade-offs. Particularly, when the queue level reaches the critical maximum capacity ($Z = 10$), the agent does not adopt a panic behavior. Instead, the action $\theta(t)$ remains smooth and bounded, confirming the pragmatic policy profile observed in the previous section.

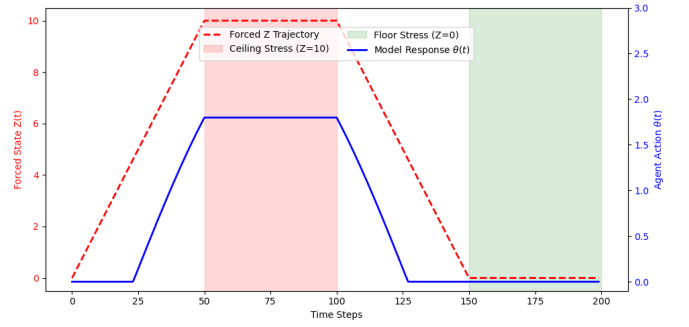


Fig. 5. Robustness test (Stress Test) at the extreme limits of the system, showing bounded and asymmetric responses.

This robustness is confirmed by the boundary stress tests (Fig. 5). Faced with trajectories forced toward the extreme limits (floor and ceiling), the agent maintains coherent and asymmetric responses, strictly respecting the nature of the Skorokhod boundaries associated with the reflected diffusion process.

D. Discussion and Limitations

These results confirm that Model-Free Deep RL is a viable alternative to classical numerical resolution methods for stochastic control, particularly thanks to the potential-based Reward Shaping mechanism.

However, an inherent limitation of this proof of concept should be noted: the current policy is specialized for a fixed set of environmental parameters (volatility, cost ratios). Any structural change in the system's economics requires retraining. Our future work will focus on augmenting the state space to include these physical parameters, thereby aiming for a universal policy. Furthermore, the algorithm's ability to overcome the curse of dimensionality paves the way for its application to multidimensional queueing networks, where closed-form analytical solutions are generally non-existent.

V. CONCLUSION

Our Deep-RL agent shows real potential to solve this kind of control problem. We do not claim a perfect success, because a true success requires an exact match with the mathematical theory. However, the agent still learned a very useful behavior. It found a practical state level to stop acting. By doing this, the agent actively saves its effort for future events.

Our tests also revealed a strong reaction to the environment settings. The entire system is highly sensitive to its physical parameters. Changing the cost values or the noise level shifts the final behavior completely. The policy we trained is currently tied to these exact numbers.

This observation defines our next research goal. We want to determine a general optimal policy. A robust model should adapt automatically when the system parameters change. Building this universal control rule will be the main focus of our future work.

REFERENCES

- [1] B. Ata, J. M. Harrison, and L. A. Shepp, “Drift rate control of a Brownian processing system,” *The Annals of Applied Probability*, vol. 15, no. 2, pp. 1145–1160, 2005.
- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [3] A. Y. Ng, D. Harada, and S. J. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [4] H. Kushner and P. Dupuis, *Numerical Methods for Stochastic Control Problems in Continuous Time*. New York, NY, USA: Springer, 2001.
- [5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [6] P. E. Kloeden and E. Platen, *Numerical Solution of Stochastic Differential Equations*. Berlin, Germany: Springer, 1992.
- [7] J. M. Harrison and M. I. Reiman, “Reflected Brownian motion on an orthant,” *The Annals of Probability*, vol. 9, no. 2, pp. 302–308, 1981.
- [8] J. M. Harrison, *Brownian Models of Performance and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2013.
- [9] J. G. Dai and M. Gluzman, “Queueing network controls via deep reinforcement learning,” *Performance Evaluation*, vol. 153, p. 102266, 2022.
- [10] T. P. Lillicrap *et al.*, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.
- [11] W. H. Fleming and H. M. Soner, *Controlled Markov Processes and Viscosity Solutions*, 2nd ed. New York, NY, USA: Springer, 2006.
- [12] D. J. Higham, “An algorithmic introduction to numerical simulation of stochastic differential equations,” *SIAM Review*, vol. 43, no. 3, pp. 525–546, 2001.
- [13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1861–1870.
- [14] S. Meyn, *Control Techniques for Complex Networks*. Cambridge, U.K.: Cambridge Univ. Press, 2008.